

Week 3: Rays and Cameras

Generating Rays, and the Default Camera

CS 4212: Computer Graphics

Fall 2026

Implement Ray Generation from a Camera

Basis for generating ray-traced or path-traced images (image-order algorithm)

- Tue - the parametric line as a representation of the ray, camera details
- Lab - implement and test the ray
- Thu - use the ray and a default camera to generate rays

The 1-Pixel Image

Draw a picture

Think of a scene: that you remember, that you can see now, or an imaginary one.

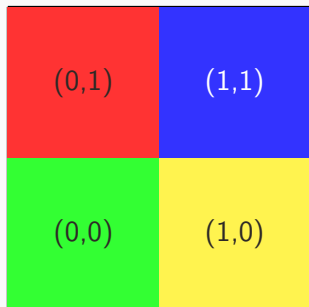


- Simplest possible image.
- If you had 1 pixel, what would the scene look like? Color it!
- **Question:** What determines this color?

The 4-Pixel Image (2x2 Grid)

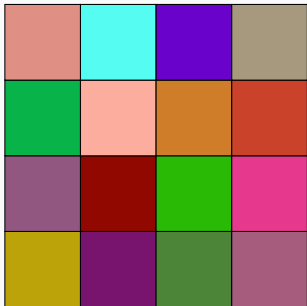
Draw a picture

Using the same scene idea...



- If you had 4 pixels, what would the scene look like? Color it!
- Coordinates (x, y) or (i, j) or (u, v) .
- Resolution: 2×2 .
- We need a loop!

The 16-Pixel Image (4x4 Grid)



If you had 16 pixels, in a 4x4 grid, what would the scene look like? Draw it!

```
for  $y = 0$  to  $height - 1$  do  
  for  $x = 0$  to  $width - 1$  do  
     $color \leftarrow ???$   
     $pixel_{x,y}.setColor(color)$   
  end for  
end for
```

The point being....

Ray tracing and, in general, rendering is about “sampling” the energy that reaches the camera.

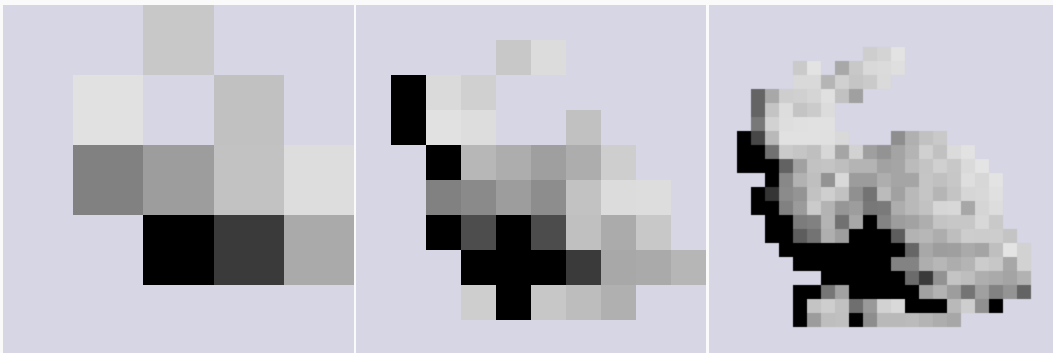
- The higher the grid resolution, the more pixels
- As you sample more pixels, you usually get a much better image

Objective of Ray Tracer

Take 3D representation and *sample* the scene to create a 2D image

Render

Stanford Bunny model



Objective of Ray Tracer

If you *sample* enough (but not too much), your image will be a good representation of the 3D information

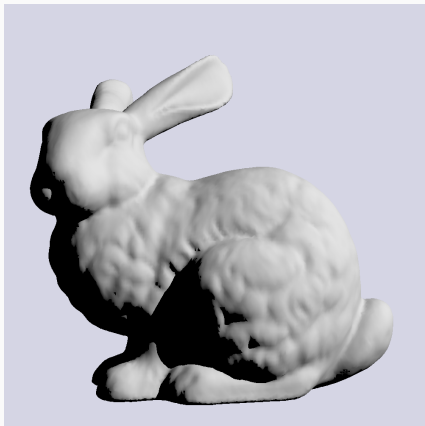
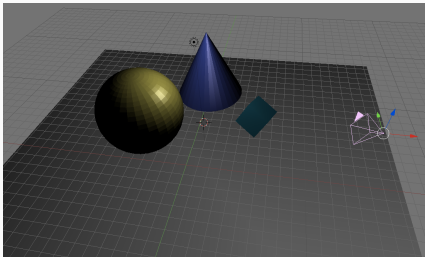
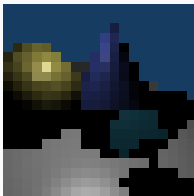


Image Generation - Image Order



```
fb  $\leftarrow$  new Framebuffer(  $n_x * n_y * 3$  )  
for all  $p_{i,j} \in \text{fb}$  do  
    ray  $\leftarrow$  computeRay(  $p_{i,j}$  )  
     $o_k \leftarrow$  findFirstObject( ray )  
    fb[ $p_{i,j}$ ]  $\leftarrow$  shader( $o_k$ )  
end for
```



Ray Tracing

What is a ray tracer?

- Mechanism for *sampling* light intensity (or more appropriately, *radiance*)
- Rays to *intersect* objects in a scene
- At point of intersection between ray and object, we sample the radiance in a small area

Radiance Characterizes the amount of light energy that is emitted or reflected from a small surface area. This is a view dependent characteristic!

Components for Image Generation - Software Engineering

Scene A basic container for everything

Shape Shapes, such as spheres, triangles, etc... that can be rendered

Shader Descriptions of the materials associated with objects in the scene

Camera Contains information about how the scene should be rendered

Light Different ways in which the scene can be lit

Scratch Coding

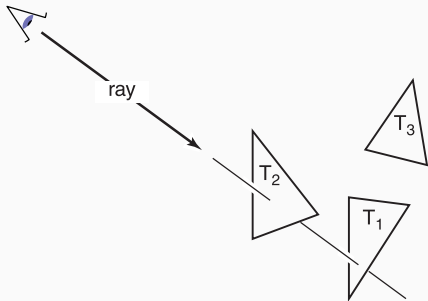
Work with a neighbor to identify which of these might be base (or abstract) classes.

What will be needed if these are abstract classes? Write down the beginning of these classes in your code.

Basic Ray Tracer

Has three major parts:

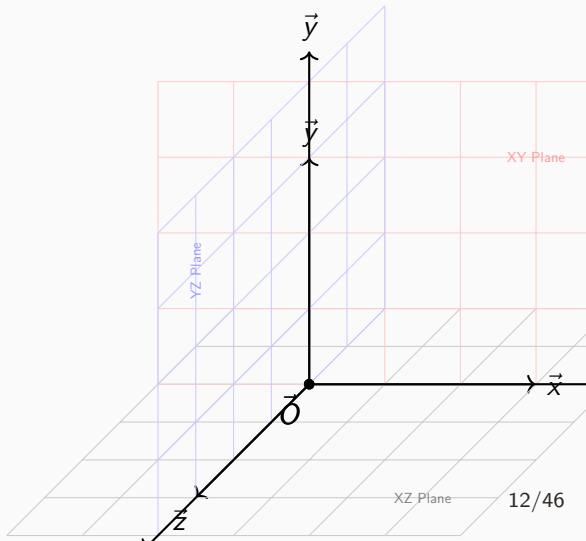
- Ray Generation
- Ray Intersection
- Shading



```
im  $\leftarrow$  new image(  $n_x n_y * 3$  )  
for all  $p_{i,j} \in \text{im}$  do  
  {Compute Viewing Ray}  
   $\text{ray}_{i,j} \leftarrow \text{computeRay}( i, j )$   
  {Intersect with Objects}  
   $o_k \leftarrow \text{findFirstObject}( \text{ray} )$   
  {Set pixel color based on object shader}  
   $\text{im}[p_{i,j}] \leftarrow \text{shader}(o_k)$   
end for
```

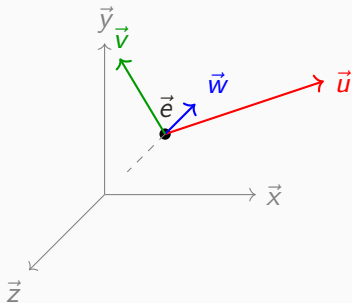
The Global Coordinate System

- The "World" usually has a fixed origin and axes.
- Often called World Space or Global Space.
- Standard Basis:
 - Origin $\vec{O} = (0, 0, 0)$
 - $\vec{x} = (1, 0, 0)$
 - $\vec{y} = (0, 1, 0)$
 - $\vec{z} = (0, 0, 1)$



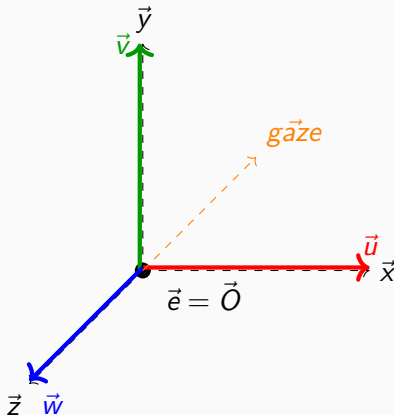
The Camera Coordinate System

- The Camera exists *inside* the World Coordinate System.
- It has its own position (Eye $\vec{e}$) and orientation (orthonormal basis).
- Orthonormal Basis: $\vec{u}$, $\vec{v}$, $\vec{w}$.
- $\vec{w}$ will point *opposite* the view direction.



The Default Camera Orientation

- The "Canonical" Camera Position is aligned with the Global System..
- Eye is at the World Origin:
 $\vec{e} = (0, 0, 0)$.
- Orthonormal bases are aligned with the World Axes:
 - $\vec{u} = \vec{x}$ (Right)
 - $\vec{v} = \vec{y}$ (Up)
 - $\vec{w} = \vec{z}$ (Back / gaze is $-\vec{z}$)
- This is the state of the camera before any transformation would be applied.



Camera Setup & Basis Vectors

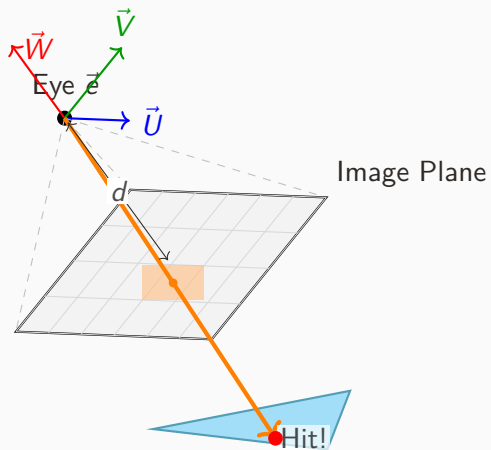
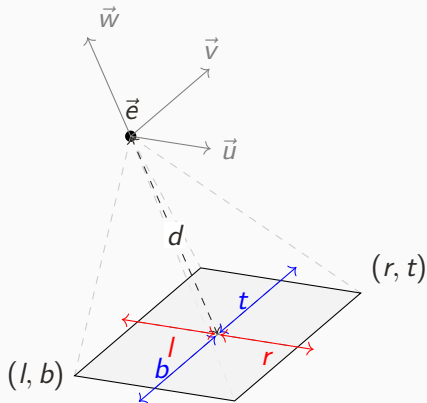


Image Plane Geometry

- Camera at Origin $\vec{e}$.
- Image Plane at distance d along $-\vec{w}$.
- Boundaries relative to center axis:
 - l : Left edge (u_{min})
 - r : Right edge (u_{max})
 - b : Bottom edge (v_{min})
 - t : Top edge (v_{max})

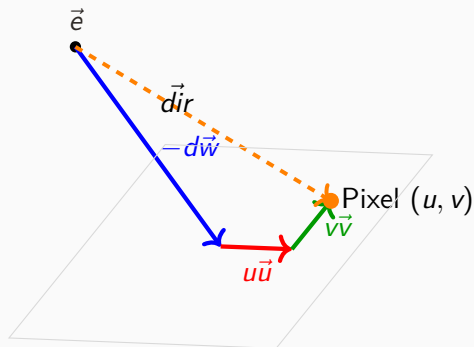


Computing the Ray Direction

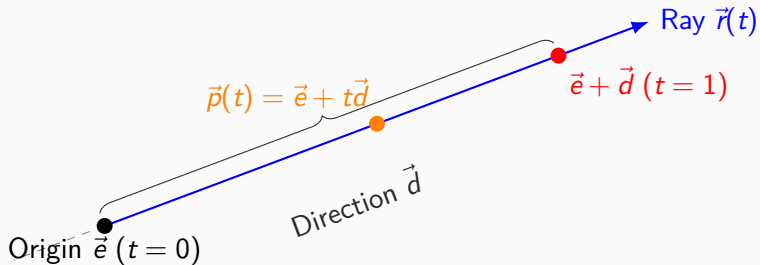
To reach pixel (u, v) :

1. Start at Eye $\vec{e}$.
2. Move distance d along $-\vec{w}$.
3. Move u along $\vec{u}$.
4. Move v along $\vec{v}$.

$$\vec{dir} = -d\vec{w} + u\vec{u} + v\vec{v}$$



Parametric Line: The Ray



- $\vec{e}$: Ray Origin (Camera Position)
- $\vec{d}$: Ray Direction (Normalized usually, but here shows $t = 1$)
- t : Parameter (Distance along ray if $\|\vec{d}\| = 1$)

Where

Parametric Ray Equation $P(t) = A + tb$ Task: Given a ray origin $A = (1, 2, 3)$ and a direction $b = (0, 1, 0)$, find the coordinates of the point at $t = -1$, $t = 0$, $t = 5$, and $t = 1,000,000$. "If this ray represents a photon, which of these values are physically impossible in our renderer, and why?"

Solve

A ray starts at $A = (0, 10, 0)$ (the camera) and points in direction $b = (0, -1, -1)$. At what value of t does the ray hit the ground plane ($y = 0$)?

Wednesday - Lab

Thursday - Putting Things Together

Goal for the Day (and rest of week)

Implement Working Camera

Camera's will be used to generate rays into the scene.

Two types of cameras we will mostly investigate:

- Perspective Camera - uses linear perspective (objects of the same size look smaller as they get farther away)
- Orthographic Camera (optional) - also known as parallel projection; preserves the relative size of objects across the scene depth; used often for CAD

You want to have a fully working Perspective Camera to use to render our scenes.

Coding Goal for the Day

```
Framebuffer fb;

// set default cam variables here
PerspectiveCamera p();
// PerspectiveCamera p( origin, viewdir, focalLength,
//      imagePlaneWidth, fb.width(), fb.height() );

for (int x=0; x<800; ++x) {
    for (int y=0; y<800; ++y) {
        Ray r;
        p.generateRay( x, y, r );

        // convert ray direction to a color here

        fb.setPixelColor(x, y, ray_dir_color);
    }
}
fb.exportAsPNG( "defaultCamRayColors.png" );
```



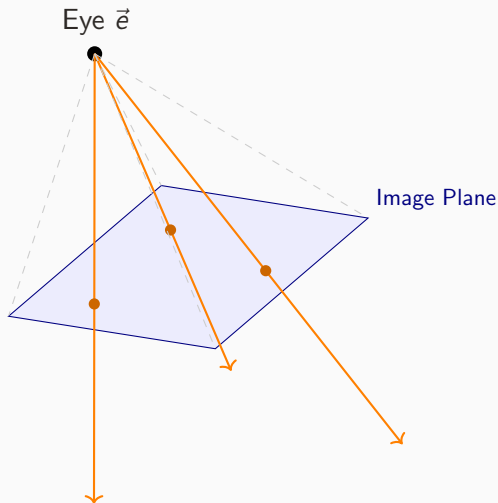
Perspective Camera

- **Origin:** All rays originate from a single point, at the camera's origin (**Eye** $\vec{e}$).
- **Direction:** Determined by the vector from the eye to the pixel center on the image plane.

$$\vec{r}_o = \vec{e}$$

$$\vec{r}_d = -d\vec{w} + u\vec{u} + v\vec{v}$$

$$\vec{p}(t) = \vec{r}_o + t\vec{r}_d$$



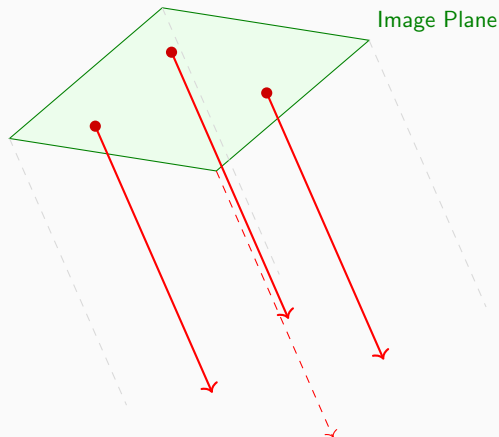
Orthographic Camera

- **Origin:** Rays originate from different points on the image plane.
- **Direction:** All rays share the same constant direction (usually $-\vec{w}$). Camera focal length does **not** affect this.

$$\vec{r}_o = \vec{e} + u\vec{u} + v\vec{v}$$

$$\vec{r}_d = -\vec{w}$$

$$\vec{p}(t) = \vec{r}_o + t\vec{r}_d$$

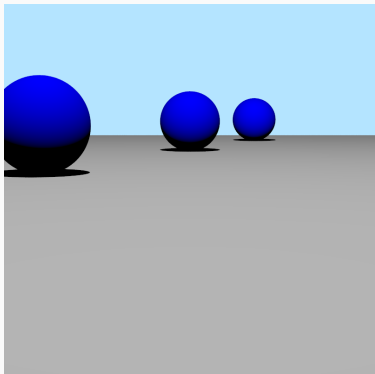


Perspective vs. Orthographic Views

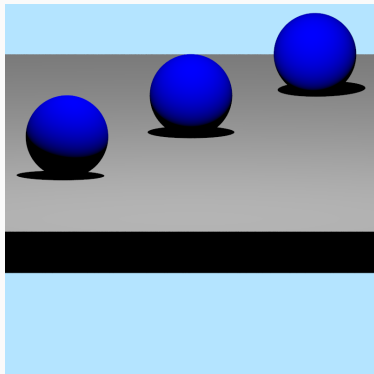
Same Scene - Three Same-size Spheres

Rendering three same size spheres on a box.

Perspective Camera



Orthographic Camera



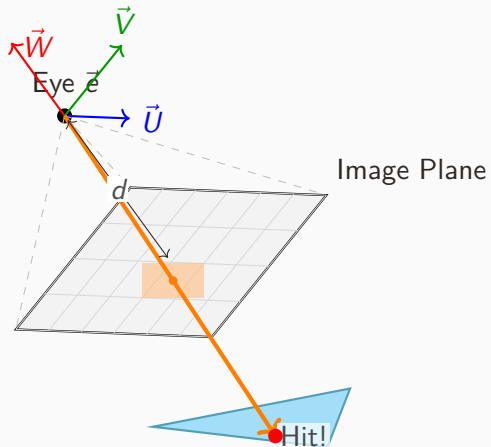
Create a Scene

What is space of the scene relative to the camera?

Working with a partner, using the default camera, create a scene with a single sphere so that the perspective camera will be able to render the entire sphere.

- Where will you place the sphere?
- Why did you place it there?

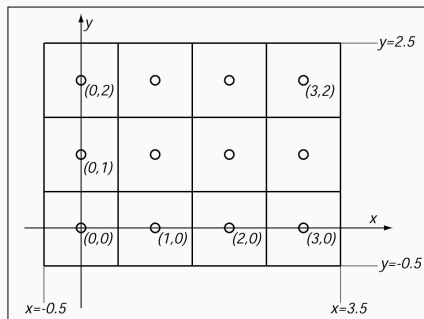
Camera Setup & Basis Vectors



Compute the Viewing Ray to Sample the Image Plane

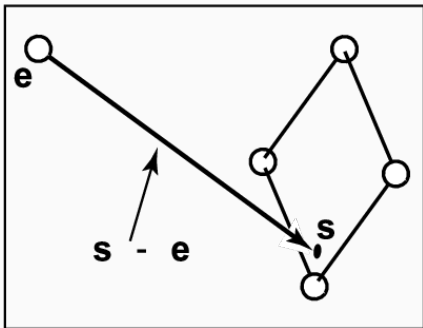
Camera needs to have information about how it samples the image plane with rays

- Image plane width and height
- Resolution of final image: n_x by n_y pixels
- Domain of pixel coords:
 $[-0.5, n_x - 0.5] \times [-0.5, n_y - 0.5]$
- Aspect ratio: width / height



Ray Tracing - Computing Rays

- What is t at different locations?
- Where is s located?



```
im ← new image(  $n_x n_y * 3$  )  
for all  $p_{i,j} \in \text{im}$  do  
  { Compute Viewing Ray }  
  ray $_{i,j}$  ← computeRay(  $i, j$  )  
  { Intersect with Objects }  
   $o_k$  ← findFirstObject( ray )  
  { Set pixel color based on object shader }  
  im[ $p_{i,j}$ ] ← shader( $o_k$ )  
end for
```

Parametric Line Equation

$$\vec{p}(t) = \vec{e} + t(\vec{s} - \vec{e})$$

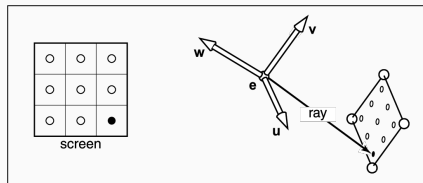
How can you easily compute $\vec{s} - \vec{e}$ using the Camera information?

Compute Ray Direction

Think about how vector math can help you compute the ray direction

Working with a partner, answer the following:

- What parameters do you have to work with for the Camera? Describe each of them.
- Using vector math how could get from the origin to center of the image plane?



Perspective Ray Generation

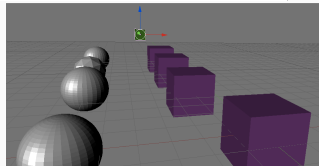
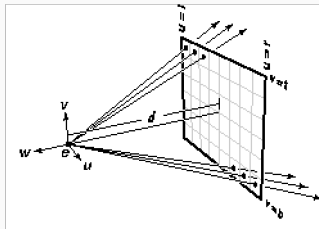
Image plane is defined in terms of

- l - left boundary ($-imageplane_{width}/2$)
- r - right boundary ($imageplane_{width}/2$)
- b - bottom boundary ($-imageplane_{height}/2$)
- t - top boundary ($imageplane_{height}/2$)

You need to calculate l , r , b and t in the camera

Scaling how far over in $\vec{u}$ and $\vec{v}$ requires fitting samples of $n_x \times n_y$ pixels into image plane rectangle of size $(r - l) \times (t - b)$

- $u = l + (r - l)(i + 0.5)/n_x$
- $v = b + (t - b)(j + 0.5)/n_y$



Perspective Ray Generation

```
im  $\leftarrow$  new image(  $n_x n_y * 3$  )  
for all  $p_{i,j} \in \text{im}$  do  
  { Compute Viewing Ray }  
   $u = l + (r - l)(i + 0.5)/n_x$   
   $v = b + (t - b)(j + 0.5)/n_y$   
  Ray r;  
  r.direction =  $-d\vec{W} + u\vec{U} + v\vec{V}$ ;  
  r.origin =  $\vec{e}$ ;  
  { Intersect with Objects }  
   $o_k \leftarrow \text{findFirstObject}( r )$   
  { Set pixel color based on object shader }  
   $\text{im}[p_{i,j}] \leftarrow \text{shader}(o_k)$   
end for
```

Code Your Cameras

Using the information in the previous slides, code the generateRay function of the Perspective Camera class. Below are useful snippets:

```
class PerspectiveCamera : public Camera {  
    public:  
        ...  
    void generateRay( int i, int j, ray &r ) override;
```

Turning a Direction into a Color

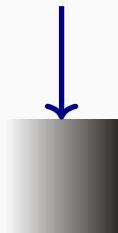
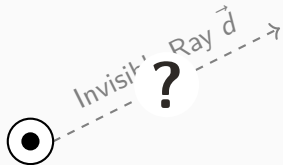
How do you turn a direction into a color?

- We calculate rays: $\vec{r}(t) = \vec{e} + t\vec{d}$.
- **Problem:** We want to visualize a direction.
- **Idea:** What if we could interpret (x, y, z) as (r, g, b) ?

In-Class Exercise

Discuss with your partner.

- What are ranges of potential values for direction vectors?
- What ranges of values do you need for colors?
- How might you map them?



Color?

Mapping Direction to Color

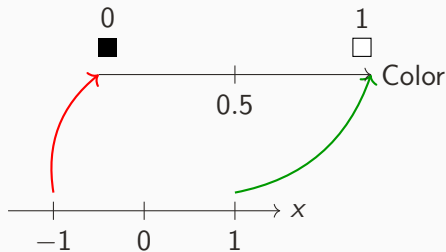
Mapping between Ranges

- Normalized Direction $\vec{d}$: Components in $[-1, 1]$.
- Color Channels: Components in $[0, 1]$.

The Solution: Remap the Range!

1. Add 1.0 $\rightarrow [0, 2]$
2. Multiply by 0.5 $\rightarrow [0, 1]$

$$\text{Color} = 0.5 \cdot (\vec{d} + 1.0)$$



$$\vec{d} = (0, 0, 1) \rightarrow (0.5, 0.5, 1.0) \text{ BLUEISH}$$

$$\vec{d} = (0, 0, -1) \rightarrow (0.5, 0.5, 0.0) \text{ YELLOWISH}$$

Coding Goal for the Day (and Weekend)

```
Framebuffer fb;

// set default cam variables here
PerspectiveCamera p();
// PerspectiveCamera p( origin, viewdir, focalLength,
//      imagePlaneWidth, fb.width(), fb.height() );

for (int x=0; x<800; ++x) {
    for (int y=0; y<800; ++y) {
        Ray r;
        p.generateRay( x, y, r );

        // convert ray direction to a color here

        fb.setPixelColor(x, y, ray_dir_color);
    }
}
fb.exportAsPNG( "defaultCamRayColors.png" );
```



Additional Camera Information Homework!

Basis Construction: Step 1

Your default camera setup is good initially.

However, you cannot render arbitrary scenes with that camera and you need to move the camera's position and it's orientation to render different views of your scenes.

You must construct your own basis vectors:

- Use additional constructors in your cameras to do this computation.

```
PerspectiveCamera p( origin, viewdir, focalLength,  
                    imagePlaneWidth, fb.width(), fb.height() );
```

Basis Construction: Step 1

Goal: create a camera basis
(U, V, W) from a view direction.

- **Input:**

- Eye Position: $\vec{e}$
- Gaze Direction: $\vec{g}$

- **Step 1:** Calculate camera back vector W .

$$W = \frac{-\vec{g}}{\|\vec{g}\|}$$

- W points opposite to where we are looking.

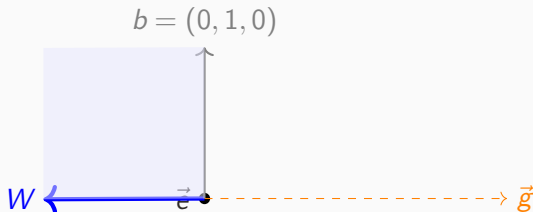


Basis Construction: Step 2

- We need a reference "up" to orient the camera.
- **Step 2:** Choose a temporary up vector b .

$$b = (0, 1, 0)$$

- Note: This fails if W is parallel to b (looking straight up or down).

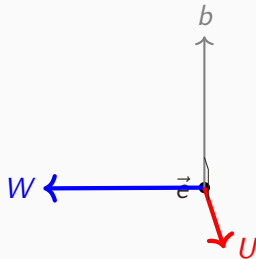


Basis Construction: Step 3

- **Step 3:** Calculate the right vector U .
- U must be perpendicular to both b and W .

$$U = \frac{b \times W}{||b \times W||}$$

- This creates a vector pointing "right" relative to our view and world up.

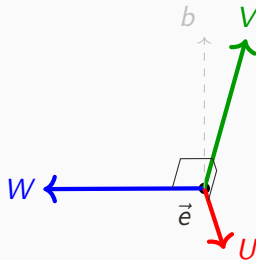


Basis Construction: Step 4

- We have W (back) and U (right). We need the true camera up vector V .
- **Step 4:** Complete the orthonormal basis.

$$V = W \times U$$

- Note: V is **not** usually $(0, 1, 0)$. It is orthogonal to W .
- Result: Frame (U, V, W) .



**More Info About Constructing a
Basis
if interested**

Creating an Orthonormal Basis

- How do you construct the basis vector for a camera?
- General mechanism when given a single vector, $\vec{a}$, construct a $\vec{U}, \vec{V}, \vec{W}$ basis
- *Requirement:* create and align $\vec{W}$ with $\vec{a}$ while not caring too much about $\vec{U}$ and $\vec{V}$
 1. $\vec{W} = \vec{a}/|\vec{a}|$
 2. Pick $\vec{t}$ such that it is NOT colinear with $\vec{W}$
 3. $\vec{U} = \vec{t} \times \vec{W} / |\vec{t} \times \vec{W}|$
 4. $\vec{V} = \vec{W} \times \vec{U}$
- Choose t such that it is W and then modify the smallest magnitude component to 1

Build your own Basis

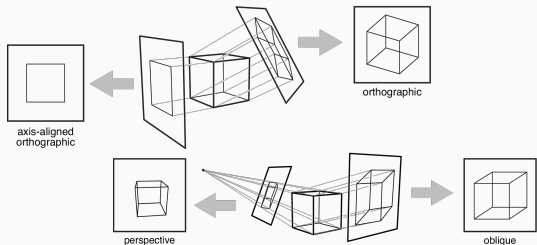
1. $\vec{W} = -\vec{a}/\|\vec{a}\|$
2. Pick $\vec{t}$ such that it is NOT colinear with $\vec{W}$
3. $\vec{U} = \vec{t} \times \vec{W} / \|\vec{t} \times \vec{W}\|$
4. $\vec{V} = \vec{W} \times \vec{U}$

- Cross Product

- $\vec{a} \times \vec{b} =$
 $(y_a z_b - z_a y_b, z_a x_b - x_a z_b, x_a y_b - y_a x_b)$

Constructing Views

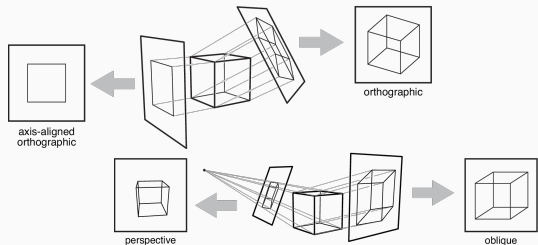
With camera basis and image coordinates, we can generate different views of the 3D representation



- Take 3D representation and “project” it on to 2D plane (the image plane)
- Many different ways to compute this projection
 - Parallel Projection
 - Perspective Projection

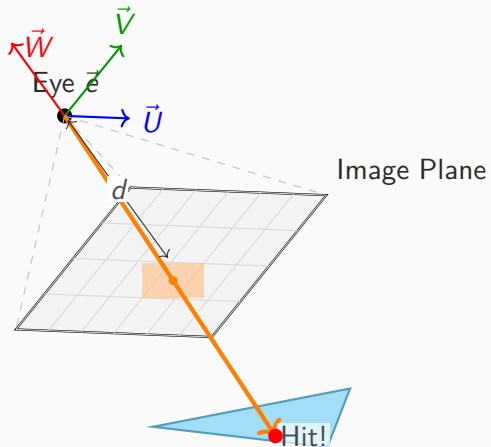
Constructing Views

With camera basis and image coordinates, we can generate different views of the 3D representation



- Take 3D representation and “project” it on to 2D plane (the image plane)
- Many different ways to compute this projection
 - Parallel Projection
 - Perspective Projection
 - Others?

Camera Setup & Basis Vectors



Homework!!

- Create Perspective Camera class with two Constructors
- Need default camera and camera that can view any direction
- Generate rays for each pixel.
- Convert ray direction to a color and store in framebuffer
- Export as PNG file
- Generate views for as many directions and field of view as you can

Field of View

How do you change field of view?